

Coins Project

Sprint 5

Sprint Goal

The goal of the sprint is to add a "template method" for coin manufacturing. And, as always, our code meets our corporate requirements: checkstyle, unit testing.

Sprint tasks

1. Write the `manufacture()` template method.
2. Write the process step methods (some are abstract, some are not).
3. Double check access (public, private, protected, final, etc.) to make sure that others who use our `Coin` class can not misuse our class.
4. Update tests as appropriate.
5. Write a `Demo` class that adds to its demonstration of our code. We should be able to choose different coin creations and see the dashboards change automatically.

Domain details

The Treasury Department undersecretary is very happy with your progress so far! But, in conversation with them they don't feel that real coins are "constructed" like we are doing. In reality, coins are "manufactured." They go through a specific manufacturing process. We are tasked with simulating this manufacturing process.

Here are the process steps.

1. Coins begin as a simple "blob" with no real resemblance to their final form.
2. Then a coin goes through a smelting process that cures the metal alloy.
3. After smelting, the coin edge is ridged if appropriate.
4. After edging, the front of the coin is imprinted with the image.
5. And then the motto is imprinted along with the year and front label.
6. After the two imprinting steps on the front, the coin is flipped.
7. After flipping the back is imprinted with the image.
8. And then the back motto is imprinted along with the value description and back label.
9. The final step is a buffing operation that smooths any microscopic edges.

Implementation details

We need a new Coin method that handles the manufacturing process; so let's call it `manufacture()`. It takes a Coin object as a parameter and returns the finished coin.

The above steps of manufacture are then taken as method calls inside the `manufacture()` method. It will look something like this:

```
public Coin manufacture(Coin c0) {  
    Coin c1 = c0.smelt(c0);  
    ...  
}
```

Notice how this simulates how the coin is input to a process step and that output is then the input to the next process step.

`Coin c0` is the penny, dime, etc. that is the result of the constructor. Previously, the constructor set all the fields of the coin, but now the constructor does not. It only sets the value, common name, and metallurgy strategy delegate. This is the initial, minimal "blob" of the coin. With this initial object we then call the `manufacture` method.

```
Coin c = new Dime();  
c = c.manufacture(c);
```

Inside the `manufacture` method are method calls to the various process steps. The `flip()` and `buff()` methods are common for all coins and so can have default implementations in the `Coin` class. For now, just have these set a boolean field to true so that we can confirm that the operation was performed.

The other methods (imprinting, ridging, etc.) are specific to the type of `Coin`. So the `Coin` will require these methods to be abstract and hence implemented in the concrete class (e.g., `Dime`). So it might look something like:

```
public class Dime {  
    ...  
    protected Coin imprintFront(Coin c) {  
        c.frontImage = "F_Roosevelt";  
        ...  
    }  
    ...  
}
```